

mini-ic-flowops 零基础大书 v3

从打开终端到能讲清项目的详细教材

生成日期：2026-06-08

这版按你的反馈重做：不再追求短，而是按零基础节奏慢慢讲。它不扩写 50 题，不加入导师追问模拟；重点加入课时化讲解、逐屏输出、文件逐段讲解、错误场景 30 例、练习题和答案、术语词典 100 条、学习顺序地图、第一遍不用学什么。

读者须知：第一遍应该怎么学

你第一遍只需要掌握主线：会进目录，会加载环境，会跑 flow，会看 run

目录，会看日志和报告，会解释 Shell/Tcl/Python 分工。不要一开始纠结 Verilog、真实 PDK、商业 EDA 工具细节。

第一遍不用学什么

- Verilog 语法细节
- 真实 PDK
- 商业 EDA 工具完整命令
- Python 全部语法
- Tcl 全部语法
- 复杂 Git 分支
- AI 大模型原理

学习顺序地图

先会 Linux 命令
-> 再会运行 flowctl
-> 再会看 runs/run_xxx
-> 再会看日志和报告
-> 再会看配置
-> 再会看 Shell/Tcl/Python
-> 再会做 QA 和比较 run
-> 再会回答面试/实习问题

第 1 课：打开终端，我现在在哪里？

本课目标：学会用 `pwd`/`ls`/`cd` 判断当前位置。

先用生活比喻理解：像在学校找教室，先知道自己在哪栋楼、哪一层。

真实公司里对应什么：实习时你要在服务器目录中找项目、找日志、找脚本。

你要运行或观察的命令

```
pwd
ls
cd mini-ic-flowops
pwd
ls
```

你应该看到什么

能看到 `bin` `config` `docs` `scripts` `runs` `reports` 等目录。

如果你看到的不一樣

如果 `cd` 失败，说明你当前目录不包含 `mini-ic-flowops`。先 `ls` 看看当前目录有什么。

对应项目文件

- 项目根目录 `mini-ic-flowops`
- `docs/linux_basics.md`

本课练习

进入项目根目录后运行 `ls`，写下你看到的 5 个目录名。

参考答案

应能写出 `bin/config/docs/scripts/runs/reports` 中至少 5 个。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 2 课：什么是项目根目录？

本课目标：知道所有相对路径从哪里开始。

先用生活比喻理解：项目根目录像一本书的封面和目录页，所有章节都从这里找。

真实公司里对应什么：公司项目通常有固定根目录，脚本以它为基准找配置和数据。

你要运行或观察的命令

```
pwd
ls README.md START_HERE.md config scripts
```

你应该看到什么

这些文件都存在，说明你在项目根目录。

如果你看到的不一樣

如果 README.md 不存在，说明你不在根目录。

对应项目文件

- README.md
- START_HERE.md
- config/
- scripts/

本课练习

为什么运行命令前要确认根目录？

参考答案

因为很多相对路径依赖项目根目录，目录错会导致找不到配置或脚本。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 3 课：什么是路径？

本课目标：理解绝对路径和相对路径。

先用生活比喻理解：绝对路径像完整地址，相对路径像从当前位置往哪走。

真实公司里对应什么：EDA 脚本经常使用路径找 RTL、SDC、报告和日志。

你要运行或观察的命令

```
pwd
ls runs/run_006/logs
cat runs/run_006/logs/flow.log
```

你应该看到什么

能看到 flow.log 内容。

如果你看到的不一樣

如果找不到 run_006，就用 flowctl status 找最新编号。

对应项目文件

- runs/run_006/logs/flow.log

本课练习

解释 runs/run_006/logs/flow.log 这个路径每一段是什么意思。

参考答案

runs 是运行结果目录，run_006 是第 6 次运行，logs 是日志目录，flow.log 是主日志。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 4 课：为什么要 chmod?

本课目标：理解 Linux 执行权限。

先用生活比喻理解：脚本像门，chmod +x 像给它开门权限。

真实公司里对应什么：服务器上脚本没有执行权限时不能运行。

你要运行或观察的命令

```
chmod +x bin/flowctl  
./bin/flowctl doctor
```

你应该看到什么

能执行 doctor，不出现 Permission denied。

如果你看到的不一樣

如果 Permission denied，重新执行 chmod +x。

对应项目文件

- bin/flowctl

本课练习

为什么 Windows 双击和 Linux 执行脚本不一样？

参考答案

Linux 有明确权限模型，可执行权限缺失时不能直接运行。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 5 课：什么是 source？

本课目标：理解环境变量为什么要加载。

先用生活比喻理解：source 像进入实验室前打开电源和工具柜。

真实公司里对应什么：EDA 环境常通过 source 脚本设置工具路径、PDK 路径、license。

你要运行或观察的命令

```
source config/env.sh
echo $FLOWOPS_ROOT
which flowctl
```

你应该看到什么

FLOWOPS_ROOT 指向项目目录，which flowctl 能找到 bin/flowctl。

如果你看到的不一样

如果 which 找不到，重新 source 或检查 config/env.sh。

对应项目文件

- config/env.sh

本课练习

source 和直接运行 ./config/env.sh 的区别是什么？

参考答案

source 会影响当前终端，普通运行只影响子进程。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 6 课：flowctl 是什么？

本课目标：知道项目统一入口。

先用生活比喻理解：flowctl 像项目遥控器，所有功能从它进入。

真实公司里对应什么：公司内部工具通常也会有统一命令入口。

你要运行或观察的命令

```
flowctl learn  
flowctl doctor
```

你应该看到什么

learn 显示学习路线，doctor 显示环境检查。

如果你看到的不一樣

如果 command not found，先 source config/env.sh。

对应项目文件

- bin/flowctl
- scripts/python/flowops/flowctl.py

本课练习

flowctl 为什么不是系统自带命令？

参考答案

它是本项目 bin/flowctl 脚本，通过 PATH 被终端找到。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 7 课：doctor 输出逐行解释

本课目标：知道环境检查结果怎么看。

先用生活比喻理解：doctor 像体检，告诉你工具是否可用。

真实公司里对应什么：入职后常先检查环境，避免工具缺失导致流程失败。

你要运行或观察的命令

```
flowctl doctor
```

你应该看到什么

看到项目根目录，以及 python3/git/tclsh 的 OK 或 WARN。

如果你看到的不一樣

tclsh WARN 不一定阻塞，项目可 fallback，但建议安装 tcl。

对应项目文件

- scripts/python/flowops/flowctl.py 中 cmd_doctor

本课练习

doctor 的目的是什么？

参考答案

先发现环境问题，不要等 flow 跑到一半才失败。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 8 课：第一次运行 testchip flow

本课目标：完整跑通岗位最相关流程。

先用生活比喻理解：像把一条小型生产线从头到尾跑一遍。

真实公司里对应什么：公司里你会经常跑 reference/testchip/internal flow。

你要运行或观察的命令

```
flowctl run --design alu --flow testchip
```

你应该看到什么

[OK] 运行完成：run_xxx，并显示日志和报告路径。

如果你看到的不一樣

失败时先看打印出的日志路径。

对应项目文件

- config/testchip_flow.yaml
- runs/run_xxx/

本课练习

这条命令的 --design 和 --flow 分别是什么意思？

参考答案

--design 选择 designs/alu，--flow 选择 config/testchip_flow.yaml。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 9 课：status 输出逐行解释

本课目标：知道如何查看历史 run。

先用生活比喻理解：像查看实验记录表，看每次实验是否成功。

真实公司里对应什么：公司里常需要找到最新 run 或失败 run。

你要运行或观察的命令

```
flowctl status
```

你应该看到什么

显示 Run ID、Status、Design、Flow、Start Time。

如果你看到的不一樣

如果没有 run，先执行 flowctl run。

对应项目文件

- runs/run_xxx/manifest.json

本课练习

status 中 PASS 说明什么？

参考答案

说明该 run 的所有配置步骤都执行成功。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 10 课：runs/run_xxx 是什么？

本课目标：理解每次运行结果隔离保存。

先用生活比喻理解：像每次实验一个独立文件夹。

真实公司里对应什么：真实项目必须保留历史结果用于回溯。

你要运行或观察的命令

```
ls runs/run_006  
ls runs/run_006/logs  
ls runs/run_006/reports  
ls runs/run_006/metrics
```

你应该看到什么

看到 logs/reports/metrics/manifest.json。

如果你看到的不一样

如果 run_006 不存在，换成 status 里最新编号。

对应项目文件

- runs/

本课练习

为什么不能每次都覆盖同一个目录？

参考答案

覆盖会丢失历史，无法比较和追溯。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 11 课：flow.log 逐段阅读

本课目标：知道失败时第一时间看什么。

先用生活比喻理解：flow.log 像流程的流水账。

真实公司里对应什么：导师让你排查失败时，首先要看主日志。

你要运行或观察的命令

```
tail -n 80 runs/run_006/logs/flow.log
```

你应该看到什么

看到 step start、step pass、run completed。

如果你看到的不一樣

如果看到 run failed，要找第一个失败步骤。

对应项目文件

- runs/run_006/logs/flow.log

本课练习

为什么先找第一个 error 而不是最后一个？

参考答案

后面的错误可能是前面失败导致的连锁反应。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 12 课：reports 目录逐个看

本课目标：知道报告是给人读的。

先用生活比喻理解：像体检报告，不同项目有不同检查单。

真实公司里对应什么：EDA 流程会产生大量 report，需要知道先看哪个。

你要运行或观察的命令

```
ls runs/run_006/reports
```

你应该看到什么

看到 synth.rpt、signoff.rpt、flow_report.html 等。

如果你看到的不一樣

如果 html 打不开，用文本文件 md/rpt 先看。

对应项目文件

- runs/run_006/reports/

本课练习

flow_report.html 和 signoff.rpt 区别是什么？

参考答案

flow_report 是总报告，signoff.rpt 是单步骤报告。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 13 课：metrics 是什么？

本课目标：区分给人看的 report 和给机器读的 metrics。

先用生活比喻理解：report 像作文，metrics 像表格分数。

真实公司里对应什么：自动比较和趋势分析需要结构化指标。

你要运行或观察的命令

```
cat runs/run_006/metrics/signoff.metrics
```

你应该看到什么

看到 key=value 格式，如 power_mw=2.7。

如果你看到的不一樣

如果没有 metrics，说明对应步骤没运行或失败。

对应项目文件

- runs/run_006/metrics/

本课练习

为什么 compare 读 metrics 而不是直接读 rpt？

参考答案

metrics 格式稳定，更适合脚本比较。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 14 课：manifest.json 逐字段理解

本课目标：理解运行身份证。

先用生活比喻理解：像实验记录单，记录谁、何时、用什么材料做了什么。

真实公司里对应什么：真实 flow 必须可追溯。

你要运行或观察的命令

```
cat runs/run_006/manifest.json
```

你应该看到什么

能看到 status/design/flow/git_commit/tool_versions/inputs/steps。

如果你看到的不一樣

如果看不懂 JSON，先只关注这几个字段。

对应项目文件

- runs/run_006/manifest.json

本课练习

manifest 中 git_commit 有什么用？

参考答案

用于追踪本次运行对应的代码版本。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 15 课：designs/alu 是什么？

本课目标：理解设计输入目录。

先用生活比喻理解：像交给生产线的原材料包。

真实公司里对应什么：实习中常需要检查 IP 包是否完整。

你要运行或观察的命令

```
ls designs/alu  
find designs/alu -maxdepth 3 -type f
```

你应该看到什么

看到 README、ip.yaml、rtl/alu.v、constraints/alu.sdc。

如果你看到的不一樣

缺文件时 precheck 或 QA 会失败。

对应项目文件

- designs/alu/

本课练习

为什么设计输入不放在 scripts 里？

参考答案

输入数据和自动化脚本职责不同，应分开管理。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 16 课：ip.yaml 逐行理解

本课目标：理解机器可读元数据。

先用生活比喻理解：像 IP 包的身份证。

真实公司里对应什么：自动化系统需要读取版本、顶层模块、语言等信息。

你要运行或观察的命令

```
cat designs/alu/ip.yaml
```

你应该看到什么

看到 name/version/owner/language/top_module。

如果你看到的不一樣

如果没有 version，QA 会 FAIL。

对应项目文件

- designs/alu/ip.yaml

本课练习

为什么 README 不够，还要 ip.yaml？

参考答案

README 给人看，ip.yaml 更适合脚本读取。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 17 课：alu.v 小白理解

本课目标：知道 RTL 是设计输入。

先用生活比喻理解：像硬件功能说明书，但用 Verilog 写。

真实公司里对应什么：真实设计流程从 RTL 和约束开始。

你要运行或观察的命令

```
cat designs/alu/rtl/alu.v
```

你应该看到什么

看到 module alu、input/output、case(op)。

如果你看到的不一樣

不懂 Verilog 细节没关系，先知道它是 RTL 输入。

对应项目文件

- designs/alu/rtl/alu.v

本课练习

第一遍要不要学完整 Verilog?

参考答案

不用，先知道它在 flow 中是输入。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 18 课：alu.sdc 小白理解

本课目标：知道约束文件作用。

先用生活比喻理解：像给工具规定考试时间和规则。

真实公司里对应什么：没有时序约束，timing 分析不可信。

你要运行或观察的命令

```
cat designs/alu/constraints/alu.sdc
```

你应该看到什么

看到 create_clock、set_input_delay、set_output_delay。

如果你看到的不一樣

不懂每条 SDC 也没关系，先知道它是时序约束。

对应项目文件

- designs/alu/constraints/alu.sdc

本课练习

为什么只有 RTL 不够？

参考答案

工具还需要知道时钟周期和输入输出约束。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 19 课：testchip_results.csv 逐列理解

本课目标：理解测试数据输入。

先用生活比喻理解：像测试成绩表，每行一个测试结果。

真实公司里对应什么：测试数据分析要先理解字段。

你要运行或观察的命令

```
head data/testchip_results.csv
```

你应该看到什么

看到 chip_id,lot,wafer,ip_name,testcase,result 等列。

如果你看到的不一樣

如果字段名改了，分析脚本可能失败。

对应项目文件

- data/testchip_results.csv

本课练习

result 和 fail_reason 分别是什么？

参考答案

result 是 PASS/FAIL，fail_reason 是失败原因。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 20 课：bin/flowctl 逐行复习

本课目标：把入口脚本和命令联系起来。

先用生活比喻理解：像遥控器按钮背后的电线。

真实公司里对应什么：内部工具入口常常是 Shell 再调用 Python/Tcl。

你要运行或观察的命令

```
cat bin/flowctl
```

你应该看到什么

看到 PROJECT_ROOT、FLOWOPS_ROOT、python3 flowctl.py。

如果你看到的不一樣

如果不懂 Bash 语法，先读注释式解释。

对应项目文件

- bin/flowctl

本课练习

flowctl run 的参数最后传给谁？

参考答案

通过 \$@ 传给 Python flowctl.py。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 21 课：config/env.sh 逐行复习

本课目标：理解环境加载。

先用生活比喻理解：像开工前设置工具箱位置。

真实公司里对应什么：公司环境里 source 脚本非常常见。

你要运行或观察的命令

```
cat config/env.sh  
source config/env.sh
```

你应该看到什么

看到 FLOWOPS_ROOT 和 flowctl 已加入 PATH。

如果你看到的不一樣

如果 source 后没效果，检查是否在项目根目录。

对应项目文件

- config/env.sh

本课练习

为什么每次新开终端要 source？

参考答案

新终端没有继承你之前手动设置的环境。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 22 课：YAML flow 配置逐行理解

本课目标：知道 flow 是可配置的。

先用生活比喻理解：像菜谱，列出做菜步骤。

真实公司里对应什么：标准流程要写成配置，方便维护和复用。

你要运行或观察的命令

```
cat config/testchip_flow.yaml
```

你应该看到什么

看到 name、description、steps。

如果你看到的不一樣

缩进错可能导致解析失败。

对应项目文件

- config/ref_flow.yaml
- config/testchip_flow.yaml

本课练习

steps 的顺序重要吗？

参考答案

重要，Python 会按顺序执行。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 23 课：Tcl synth.tcl 逐段理解

本课目标：知道 Tcl 如何生成报告。

先用生活比喻理解：像某个工位上的工人，接收输入并写结果。

真实公司里对应什么：真实 EDA Tcl 控制工具执行综合、报告等步骤。

你要运行或观察的命令

```
cat scripts/tcl/synth.tcl
```

你应该看到什么

看到 argv、glob、open、puts、close。

如果你看到的不一樣

不懂 Tcl 细节时，先分成读参数、算指标、写文件三段。

对应项目文件

- scripts/tcl/synth.tcl

本课练习

metrics_path 和 report_path 从哪里来？

参考答案

Python run_tcl_step 调用 Tcl 时传入。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 24 课：Python 故事线 1：命令如何被识别？

本课目标：理解 argparse 分发。

先用生活比喻理解：像前台根据你的业务把你带到对应窗口。

真实公司里对应什么：内部 CLI 工具都需要命令解析。

你要运行或观察的命令

```
grep -n "build_parser" scripts/python/flowops/flowctl.py
```

你应该看到什么

能找到 build_parser 和 sub.add_parser。

如果你看到的不一樣

不用理解全部 argparse，先知道命令和函数的对应关系。

对应项目文件

- scripts/python/flowops/flowctl.py

本课练习

run 命令对应哪个函数？

参考答案

cmd_run。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 25 课：Python 故事线 2：run 编号如何生成？

本课目标：理解每次运行独立编号。

先用生活比喻理解：像实验编号自动递增。

真实公司里对应什么：真实 flow 会保存每次运行记录。

你要运行或观察的命令

```
grep -n "def next_run_id" scripts/python/flowops/flowctl.py
```

你应该看到什么

能找到 run_001/run_002 生成逻辑。

如果你看到的不一樣

如果 runs 里已有 run，编号会继续增加。

对应项目文件

- runs/
- flowctl.py next_run_id

本课练习

为什么编号不能每次都 run_001？

参考答案

会覆盖历史，无法追溯。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 26 课：Python 故事线 3：配置如何读取？

本课目标：理解 flow yaml 进入 Python。

先用生活比喻理解：像程序先读菜谱再做菜。

真实公司里对应什么：流程步骤应配置化而非硬编码。

你要运行或观察的命令

```
grep -n "load_simple_yaml" scripts/python/flowops/flowctl.py
```

你应该看到什么

能看到简单 YAML 解析逻辑。

如果你看到的不一樣

如果 YAML 格式复杂，当前简单解析器不一定支持。

对应项目文件

- config/testchip_flow.yaml

本课练习

为什么本项目不用复杂 YAML 库？

参考答案

为了零依赖教学，用简单格式就够。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 27 课：Python 故事线 4：precheck 怎么检查？

本课目标：理解输入检查逻辑。

先用生活比喻理解：像出门前检查钥匙、手机、钱包。

真实公司里对应什么：真实 flow 会先检查输入和环境。

你要运行或观察的命令

```
grep -n "def precheck" scripts/python/flowops/flowctl.py
```

你应该看到什么

看到检查 RTL、SDC、ip.yaml、README。

如果你看到的不一樣

precheck 失败会停止流程。

对应项目文件

- runs/run_006/reports/precheck.md

本课练习

为什么 precheck 失败不继续跑？

参考答案

后续步骤依赖完整输入，继续跑会浪费时间。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 28 课：Python 故事线 5：如何调用 Tcl?

本课目标：理解 Python 和 Tcl 协作。

先用生活比喻理解：Python 像调度员，Tcl 像具体工具工位。

真实公司里对应什么：真实流程常由 Python/Shell 调用 EDA Tcl。

你要运行或观察的命令

```
grep -n "def run_tcl_step" scripts/python/flowops/flowctl.py
```

你应该看到什么

看到 tclsh 判断和 fallback。

如果你看到的不一樣

没有 tclsh 会用 Python 模拟。

对应项目文件

- scripts/tcl/*.tcl

本课练习

为什么要有 fallback?

参考答案

让没有 Tcl 的新手也能先跑通教学流程。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 29 课：Python 故事线 6：IP QA 如何生成报告？

本课目标：理解自动检查交付包。

先用生活比喻理解：像交作业前按清单检查。

真实公司里对应什么：实习生常被安排做交付包检查自动化。

你要运行或观察的命令

```
grep -n "def run_ip_qa" scripts/python/flowops/flowctl.py
```

你应该看到什么

看到 README/ip.yaml/RTL/SDC/version 等检查。

如果你看到的不一樣

FAIL 时打开 HTML/MD 报告。

对应项目文件

- reports/alu_standalone_ip_qa/

本课练习

为什么 QA 同时输出 json/md/html？

参考答案

机器、开发者和读者需要不同格式。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 30 课：Python 故事线 7：测试数据怎么分析？

本课目标：理解 CSV 到报告。

先用生活比喻理解：像把成绩表统计成通过率和薄弱项。

真实公司里对应什么：测试芯片数据分析要找失败集中点。

你要运行或观察的命令

```
grep -n "def analyze_testchip_data" scripts/python/flowops/flowctl.py
```

你应该看到什么

看到 `csv.DictReader` 和 `Counter`。

如果你看到的不一樣

字段缺失会导致分析失败。

对应项目文件

- `data/testchip_results.csv`
- `runs/run_006/reports/testchip_analysis.md`

本课练习

为什么统计 `fail_by_ip`?

参考答案

帮助判断哪个 IP 最可能有问题。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 31 课：Python 故事线 8：manifest 怎么记录？

本课目标：理解可追溯性。

先用生活比喻理解：像实验记录单，记录材料、工具、时间和结果。

真实公司里对应什么：真实流程必须能复现和解释结果变化。

你要运行或观察的命令

```
cat runs/run_006/manifest.json
```

你应该看到什么

看到 git_commit、tool_versions、inputs、steps。

如果你看到的不一樣

如果没有 commit，说明项目不在 Git 仓库或 Git 不可用。

对应项目文件

- manifest.json

本课练习

manifest 最重要的四类信息是什么？

参考答案

代码版本、工具版本、输入文件、步骤状态。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 32 课：制造 QA 失败

本课目标：亲手体验错误。

先用生活比喻理解：故意拿走作业封面，看检查系统能否发现。

真实公司里对应什么：QA 自动化需要能准确发现交付问题。

你要运行或观察的命令

```
cp -r designs/alu designs/badip
rm designs/badip/README.md
flowctl qa --design badip
```

你应该看到什么

QA 应该 FAIL，提示 README 缺失。

如果你看到的不一樣

如果没有 FAIL，检查你是否真的删除了 badip/README.md。

对应项目文件

- designs/badip
- reports/badip_standalone_ip_qa

本课练习

为什么这个实验重要？

参考答案

它让你从使用者变成排错者。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 33 课：修复 QA 失败

本课目标：体验修复闭环。

先用生活比喻理解：补回作业封面再重新检查。

真实公司里对应什么：排错不只是发现问题，还要验证修复。

你要运行或观察的命令

```
cp designs/alu/README.md designs/badip/README.md
flowctl qa --design badip
rm -rf designs/badip
```

你应该看到什么

QA 应该 PASS。

如果你看到的不一樣

如果仍然 FAIL，打开报告看是否还有其他缺失。

对应项目文件

- reports/badip_standalone_ip_qa

本课练习

为什么修复后要重跑 QA？

参考答案

因为你需要证据证明问题已解决。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 34 课：比较两次 run

本课目标：体验结果对比。

先用生活比喻理解：像比较两次考试分数，看哪些科目变了。

真实公司里对应什么：真实项目常要解释面积/功耗/时序变化。

你要运行或观察的命令

```
flowctl run --design alu --flow ref
flowctl run --design alu --flow ref
flowctl status
flowctl compare run_001 run_002
```

你应该看到什么

compare 输出每个 metric 的 A/B/Delta。

如果你看到的不一樣

编号要换成你自己的最新两个 ref run。

对应项目文件

- runs/*/metrics

本课练习

如果两次完全一样说明什么？

参考答案

输入和逻辑一致时，模拟指标没有变化。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 35 课：归档 run

本课目标：理解交付和留档。

先用生活比喻理解：像把一次实验记录打包交给老师。

真实公司里对应什么：真实工作中会把日志和报告打包给导师或系统。

你要运行或观察的命令

```
flowctl archive --run latest  
ls reports
```

你应该看到什么

看到 reports/run_xxx.zip。

如果你看到的不一樣

如果没有 latest run，先跑一次 flow。

对应项目文件

- reports/run_xxx.zip

本课练习

归档里应该包含什么？

参考答案

日志、报告、metrics、manifest。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 36 课：用 ask 查文档

本课目标：理解内部知识库雏形。

先用生活比喻理解：像在项目说明书里搜索答案。

真实公司里对应什么：内部 AI 工具常先做文档检索和问答入口。

你要运行或观察的命令

```
flowctl ask "怎么看日志"  
flowctl ask "manifest 是什么"
```

你应该看到什么

返回相关 docs 文件。

如果你看到的不一樣

如果没结果，换更短关键词。

对应项目文件

- docs/
- cmd_ask

本课练习

flowctl ask 是大模型吗？

参考答案

不是，是关键词检索，模拟知识库入口。

本课小结

- 先确认自己在正确目录，再运行命令。
- 看到输出后，不要急着往下跑，先判断它是否符合预期。
- 命令、输出、文件、代码要连起来理解。

第 37 课：文件讲解 - bin/flowctl

这个文件是什么：Linux 命令入口脚本

谁会读取它：用户和终端先执行它

什么时候会用它：每次输入 flowctl 时

文件内容

```
#!/usr/bin/env bash
set -euo pipefail

PROJECT_ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
export FLOWOPS_ROOT="$PROJECT_ROOT"

python3 "$PROJECT_ROOT/scripts/python/flowops/flowctl.py" "$@"
```

第一段

声明 bash 并启用严格模式。

第二段

计算项目根目录并设置 FLOWOPS_ROOT。

第三段

调用 Python 主程序并传递参数。

真实公司里对应什么

真实内部工具也常用 Shell 包一层 Python/Tcl。

本课练习

打开 `bin/flowctl`，用自己的话解释它在项目流程图里的位置。

第 38 课：文件讲解 - config/env.sh

这个文件是什么：环境加载脚本

谁会读取它：用户通过 source 加载

什么时候会用它：每次新开终端后

文件内容

```
#!/usr/bin/env bash

# 用法：
# source config/env.sh
#
# 这类文件在真实公司环境里很常见，用来设置工具路径、license 路径、项目路径等。

export FLOWOPS_ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
export PATH="$FLOWOPS_ROOT/bin:$PATH"

echo "FLOWOPS_ROOT=$FLOWOPS_ROOT"
echo "flowctl 已加入 PATH，可以直接运行：flowctl doctor"
```

环境变量

FLOWOPS_ROOT 记录项目根目录。

PATH

把 bin 加入命令搜索路径。

提示输出

帮助新手确认 source 成功。

真实公司里对应什么

公司环境 setup 脚本会更复杂，包含 EDA 工具、license、PDK。

本课练习

打开 `config/env.sh`，用自己的话解释它在项目流程图里的位置。

第 39 课：文件讲解 - config/ref_flow.yaml

这个文件是什么：Reference Flow 配置

谁会读取它：Python load_simple_yaml 读取

什么时候会用它：运行 --flow ref 时

文件内容

```
# Reference Flow: 基础参考流程
name: ref
description: "基础 RTL 到 signoff 的模拟参考流程"
steps:
  - precheck
  - synth
  - floorplan
  - place
  - route
  - signoff
  - report
```

name

流程名称。

description

人类可读说明。

steps

按顺序执行的步骤列表。

真实公司里对应什么

Reference Flow 是团队复用的标准流程模板。

本课练习

打开 `config/ref\_flow.yaml`，用自己的话解释它在项目流程图里的位置。

第 40 课：文件讲解 - config/testchip_flow.yaml

这个文件是什么：Testchip Flow 配置

谁会读取它：Python 主流程读取

什么时候会用它：运行 --flow testchip 时

文件内容

```
# Testchip Flow: 在基础流程后增加 IP QA 和测试数据收集
name: testchip
description: "模拟 testchip 设计、QA、测试数据分析的完整流程"
steps:
  - precheck
  - synth
  - floorplan
  - place
  - route
  - signoff
  - ip_qa
  - data_collect
  - report
```

基础步骤

precheck 到 signoff。

扩展步骤

ip_qa 和 data_collect。

report

最终汇总输出。

真实公司里对应什么

Testchip Flow 常比基础 flow 多测试和质量检查环节。

本课练习

打开 `config/testchip\_flow.yaml`，用自己的话解释它在项目流程图里的位置。

第 41 课：文件讲解 - designs/alu/ip.yaml

这个文件是什么：IP 元数据

谁会读取它：QA 脚本读取/检查

什么时候会用它：precheck 和 qa 时

文件内容

```
name: alu
version: 0.1.0
owner: training
language: verilog
top_module: alu
description: "用于 mini-ic-flowops 的示例 ALU IP"
```

name/version

标识 IP 和版本。

language/top_module

说明设计语言和顶层模块。

真实公司里对应什么

真实 IP 交付中元数据可被平台读取。

本课练习

打开 `designs/alu/ip.yaml`，用自己的话解释它在项目流程图里的位置。

第 42 课：文件讲解 - designs/alu/rtl/alu.v

这个文件是什么：示例 RTL

谁会读取它：EDA flow 输入

什么时候会用它：synth 之前

文件内容

```
// Simple ALU for flow automation practice.
module alu (
    input wire [7:0] a,
    input wire [7:0] b,
    input wire [2:0] op,
    output reg [7:0] y,
    output wire    zero
);

assign zero = (y == 8'b0);

always @(*) begin
    case (op)
        3'b000: y = a + b;
        3'b001: y = a - b;
        3'b010: y = a & b;
        3'b011: y = a | b;
        3'b100: y = a ^ b;
        3'b101: y = b << 1;
        3'b110: y = b >> 1;
        default: y = 8'b0;
    endcase
end

endmodule
```

module alu

定义 ALU 模块。

input/output

定义接口。

case(op)

按 op 选择运算。

真实公司里对应什么

真实 RTL 会复杂得多，但在 flow 中同样是输入。

本课练习

打开 `designs/alu/rtl/alu.v`，用自己的话解释它在项目流程图里的位置。

第 43 课：文件讲解 - designs/alu/constraints/alu.sdc

这个文件是什么：时序约束

谁会读取它：EDA 工具读取

什么时候会用它：综合和签核时

文件内容

```
# Simplified timing constraint for practice.  
create_clock -name clk -period 10.0  
set_input_delay 1.0 -clock clk [all_inputs]  
set_output_delay 1.0 -clock clk [all_outputs]
```

create_clock

定义时钟。

input/output delay

定义输入输出延迟。

真实公司里对应什么

真实 SDC 是 timing 是否可信的关键。

本课练习

打开 `designs/alu/constraints/alu.sdc`，用自己的话解释它在项目流程图里的位置。

第 44 课：文件讲解 - scripts/tcl/synth.tcl

这个文件是什么：综合步骤模拟脚本

谁会读取它：Python 调用 tclsh 执行

什么时候会用它：synth step

文件内容

```
# Tcl step: synth
set run_dir [lindex $argv 0]
set design_dir [lindex $argv 1]
set metrics_path [lindex $argv 2]
set report_path [lindex $argv 3]

set rtl_files [glob -nocomplain -directory "$design_dir/rtl" *.v]
set rtl_count [llength $rtl_files]
set cell_count [expr {120 + $rtl_count * 17}]
set area_um2 [expr {$cell_count * 6.25}]
set wns_ns 0.18

set fp [open $metrics_path w]
puts $fp "step=synth"
puts $fp "rtl_files=$rtl_count"
puts $fp "cell_count=$cell_count"
puts $fp "area_um2=$area_um2"
puts $fp "wns_ns=$wns_ns"
close $fp

set rp [open $report_path w]
puts $rp "SYNTHESIS REPORT"
puts $rp "Design dir: $design_dir"
puts $rp "RTL files: $rtl_count"
puts $rp "Cell count: $cell_count"
puts $rp "Area um2: $area_um2"
puts $rp "WNS ns: $wns_ns"
close $rp

puts "synth completed"
```

argv

接收路径参数。

metrics

写 key=value 指标。

report

写人类可读报告。

真实公司里对应什么

真实综合 Tcl 会调用商业工具命令。

本课练习

打开 `scripts/tcl/synth.tcl`，用自己的话解释它在项目流程图里的位置。

第 45 课：文件讲解 - scripts/tcl/signoff.tcl

这个文件是什么：签核步骤模拟脚本

谁会读取它：Python 调用

什么时候会用它：signoff step

文件内容

```
# Tcl step: signoff
set run_dir [lindex $argv 0]
set design_dir [lindex $argv 1]
set metrics_path [lindex $argv 2]
set report_path [lindex $argv 3]

set setup_wns_ns 0.11
set hold_wns_ns 0.04
set lvs_errors 0
set drc_errors 0
set power_mw 2.7

set fp [open $metrics_path w]
puts $fp "step=signoff"
puts $fp "setup_wns_ns=$setup_wns_ns"
puts $fp "hold_wns_ns=$hold_wns_ns"
puts $fp "lvs_errors=$lvs_errors"
puts $fp "drc_errors=$drc_errors"
puts $fp "power_mw=$power_mw"
close $fp

set rp [open $report_path w]
puts $rp "SIGNOFF REPORT"
puts $rp "Setup WNS ns: $setup_wns_ns"
puts $rp "Hold WNS ns: $hold_wns_ns"
puts $rp "LVS errors: $lvs_errors"
puts $rp "DRC errors: $drc_errors"
puts $rp "Power mW: $power_mw"
close $rp

puts "signoff completed"
```

setup/hold

时序指标。

DRC/LVS

物理验证指标。

power

功耗指标。

真实公司里对应什么

真实 signoff 是流片前重要质量关口。

本课练习

打开 `scripts/tcl/signoff.tcl`，用自己的话解释它在项目流程图里的位置。

第 46 课：文件讲解 - data/testchip_results.csv

这个文件是什么：测试数据表

谁会读取它：Python CSV 分析读取

什么时候会用它：data_collect step

文件内容

```
chip_id,lot,wafer,die_x,die_y,ip_name,testcase,voltage,freq,result,fail_reason
TC001,L01,W01,0,0,alu,add_basic,0.80,100,PASS,
TC002,L01,W01,0,1,alu,sub_basic,0.80,100,PASS,
TC003,L01,W01,0,2,alu,and_or,0.80,100,PASS,
TC004,L01,W01,1,0,alu,shift_left,0.80,100,FAIL,timing_margin
TC005,L01,W01,1,1,alu,shift_right,0.80,100,PASS,
TC006,L01,W01,1,2,alu,add_basic,0.72,120,FAIL,low_voltage
TC007,L01,W02,0,0,alu,sub_basic,0.72,120,PASS,
TC008,L01,W02,0,1,alu,and_or,0.72,120,PASS,
TC009,L01,W02,0,2,alu,shift_left,0.72,120,FAIL,timing_margin
TC010,L01,W02,1,0,alu,shift_right,0.72,120,PASS,
TC011,L02,W01,0,0,timer,reset_basic,0.80,100,PASS,
TC012,L02,W01,0,1,timer,counter_rollover,0.80,100,FAIL,functional
TC013,L02,W01,0,2,timer,interrupt,0.80,100,PASS,
TC014,L02,W01,1,0,gpio,input_sample,0.80,100,PASS,
TC015,L02,W01,1,1,gpio,output_drive,0.80,100,PASS,
TC016,L02,W02,0,0,gpio,input_sample,0.72,120,PASS,
TC017,L02,W02,0,1,gpio,output_drive,0.72,120,FAIL,signal_integrity
TC018,L02,W02,0,2,alu,add_basic,0.90,150,PASS,
TC019,L02,W02,1,0,alu,shift_left,0.90,150,FAIL,timing_margin
TC020,L02,W02,1,1,timer,counter_rollover,0.90,150,PASS,
```

每行

一个测试样本。

result

PASS/FAIL。

fail_reason

失败原因。

真实公司里对应什么

真实测试数据会更大，需要清洗、分组和可视化。

本课练习

打开 `data/testchip\_results.csv`，用自己的话解释它在项目流程图里的位置。

第 47 课：文件讲解 - runs/run_006/manifest.json

这个文件是什么：运行身份证

谁会读取它：人和脚本都可读取

什么时候会用它：每次 run 结束后

文件内容

```
{
  "run_id": "run_006",
  "status": "PASS",
  "design": "alu",
  "flow": "testchip",
  "flow_config": "config/testchip_flow.yaml",
  "start_time": "2026-06-08 14:28:22",
  "end_time": "2026-06-08 14:28:22",
  "git_commit": "024082d",
  "tool_versions": {
    "python3": "missing",
    "git": "git version 2.54.0.windows.1",
    "tclsh": "missing"
  },
  "inputs": {
    "files": [
      {
        "path": "designs/alu/constraints/alu.sdc",
        "sha256": "def01c7fc4453e59b7c97d4f65446f6156a9dfcbc82cb9402de95628b0a017b3"
      },
      {
        "path": "designs/alu/docs/interface.md",
        "sha256": "e10b474858a13092b6a62c475a50434d27b89528e6601d553be1d62ffdc21ca4"
      },
      {
        "path": "designs/alu/ip.yaml",
        "sha256": "6ab3f4ed3e74a860724b115fe5a1258a7038a8228650527b211e5738db3991ec"
      },
      {
        "path": "designs/alu/README.md",
        "sha256": "2eb81a9ca924a30fec5c8a520a5f402ccd8e4914e3dbdf31e569e2de2a3c3c77"
      },
      {
        "path": "designs/alu/rtl/alu.v",
        "sha256": "8e18967958dd62dd3115e1b7f6f814b8026f326490802695d76d5b796ba031fb"
      },
      {
        "path": "config/testchip_flow.yaml",
        "sha256": "32537284622c9176f6939534593d0a2e037103fa605554ec566b1670587ae9af"
      }
    ]
  },
  "steps": [
    {
      "name": "precheck",
      "status": "PASS",
      "seconds": 0.003
    },
    {
```

```
    "name": "synth",
    "status": "PASS",
    "seconds": 0.004
  },
  {
    "name": "floorplan",
    "status": "PASS",
    "seconds": 0.003
  },
  {
    "name": "place",
    "status": "PASS",
    "seconds": 0.004
  },
  {
    "name": "route",
    "status": "PASS",
    "seconds": 0.004
  },
  {
    "name": "signoff",
    "status": "PASS",
    "seconds": 0.004
  },
  {
    "name": "ip_qa",
    "status": "PASS",
    "seconds": 0.004
  },
  {
    "name": "data_collect",
    "status": "PASS",
    "seconds": 0.002
  },
  {
    "name": "report",
    "status": "PASS",
    "seconds": 0.012
  }
]
}
```

status/design/flow

基本身份。

git_commit

代码版本。

inputs.files

输入文件和 hash。

steps

步骤状态和耗时。

真实公司里对应什么

真实 flow 必须有可追溯记录。

本课练习

打开 ``runs/run_006/manifest.json``，用自己的话解释它在项目流程图里的位置。

练习 1：确认自己在项目根目录

练习目标：不依赖别人提醒，自己判断当前目录是否正确。

操作步骤

```
pwd  
ls
```

你应该看到什么

能看到 README.md、START_HERE.md、bin、config、scripts、docs。

标准答案/解释

如果这些文件和目录存在，说明你在项目根目录。后续相对路径命令都以这里为起点。

进阶思考

如果你在别的目录，哪些命令会失败？为什么？

练习 2：解释 flowctl doctor 输出

练习目标：看到环境检查结果后知道是否能继续。

操作步骤

```
source config/env.sh  
flowctl doctor
```

你应该看到什么

项目根目录正确，Python/Git/Tcl 至少 Python 和 Git 可用。

标准答案/解释

doctor 是开跑前体检。Tcl 缺失时本项目可 fallback，但真实 EDA Tcl 练习建议安装 tcl。

进阶思考

如果 Python 缺失，为什么 fallback 也无法工作？

练习 3：完整跑一次 testchip flow

练习目标：跑出完整结果。

操作步骤

```
flowctl run --design alu --flow testchip  
flowctl status
```

你应该看到什么

最新 run 状态为 PASS。

标准答案/解释

PASS 表示配置中的每一步都成功执行，生成了日志、报告、metrics 和 manifest。

进阶思考

如果只是 ref flow，输出会少哪些报告？

练习 4：找到最新 run 的日志

练习目标：学会从 status 到日志路径。

操作步骤

```
flowctl status  
ls runs  
tail -n 80 runs/run_006/logs/flow.log
```

你应该看到什么

能看到 step start、step pass、run completed。

标准答案/解释

日志是排错第一入口。run 编号要以你机器上的最新编号为准。

进阶思考

为什么看日志最后 80 行通常比从头看更高效？

练习 5：找到最新 run 的总报告

练习目标：知道给人看的报告在哪里。

操作步骤

```
ls runs/run_006/reports  
cat runs/run_006/reports/flow_report.md
```

你应该看到什么

能看到 Step Metrics、IP QA、Testchip Analysis。

标准答案/解释

flow_report 是汇总报告，适合先给导师看；单步骤问题再看对应 rpt。

进阶思考

如果 HTML 打不开，Markdown 报告有什么价值？

练习 6：读取 signoff 指标

练习目标：理解 metrics key=value。

操作步骤

```
cat runs/run_006/metrics/signoff.metrics
```

你应该看到什么

看到 setup_wns_ns、hold_wns_ns、lvs_errors、drc_errors、power_mw。

标准答案/解释

metrics 是结构化指标，适合 compare 和脚本分析。

进阶思考

为什么 signoff 比 synth 更接近最终质量判断？

练习 7：解释 manifest 的四个核心字段

练习目标：从 manifest 里抓重点。

操作步骤

```
cat runs/run_006/manifest.json
```

你应该看到什么

能找到 git_commit、tool_versions、inputs.files、steps。

标准答案/解释

这四类信息分别回答：代码版本是什么、工具环境是什么、输入是什么、每步结果是什么。

进阶思考

如果两次 run 的 git_commit 不同，结果变化可能来自哪里？

练习 8：看懂 testchip_flow.yaml

练习目标：理解流程来自配置。

操作步骤

```
cat config/testchip_flow.yaml
```

你应该看到什么

看到 steps 列表。

标准答案/解释

Python 按 steps 顺序执行。ip_qa 和 data_collect 是 testchip flow 比 ref flow 多出的部分。

进阶思考

如果把 report 放到最前面会发生什么？

练习 9：解释 Shell 入口

练习目标：知道 flowctl 如何进入 Python。

操作步骤

```
cat bin/flowctl
```

你应该看到什么

看到 PROJECT_ROOT、FLOWOPS_ROOT、python3 flowctl.py "\$@"。

标准答案/解释

Shell 入口只做启动和转发，复杂逻辑在 Python。

进阶思考

为什么不直接让用户每次输入 python3 scripts/python/...？

练习 10：解释 Tcl synth 脚本三段结构

练习目标：把 Tcl 文件分成读参数、写 metrics、写 report。

操作步骤

```
cat scripts/tcl/synth.tcl
```

你应该看到什么

能指出 argv、open metrics、open report。

标准答案/解释

Tcl 脚本模拟 EDA 工具步骤。真实工具命令将替换模拟指标。

进阶思考

如果 Tcl 报告路径写错，会影响哪些输出？

练习 11：制造 QA 失败

练习目标：亲手看到 FAIL。

操作步骤

```
cp -r designs/alu designs/badip
rm designs/badip/README.md
flowctl qa --design badip
```

你应该看到什么

QA 状态 FAIL，报告提示 README 缺失。

标准答案/解释

这是交付包不完整的典型错误。QA 的价值是早发现、自动发现。

进阶思考

如果缺的是 SDC，影响会更偏向哪个流程？

练习 12：修复 QA 失败

练习目标：完成发现-修复-验证闭环。

操作步骤

```
cp designs/alu/README.md designs/badip/README.md
flowctl qa --design badip
rm -rf designs/badip
```

你应该看到什么

QA 状态 PASS。

标准答案/解释

修复后一定要重跑 QA，不能只凭感觉认为好了。

进阶思考

真实工作中为什么修复后要留下证据？

练习 13：比较两个 run

练习目标：理解结果差异表。

操作步骤

```
flowctl status  
flowctl compare run_001 run_002
```

你应该看到什么

显示 Metric、A、B、Delta。

标准答案/解释

Delta 是 B-A。没有差异通常说明输入和模拟逻辑一致。

进阶思考

如果 power_mw 变化，应该去查哪些文件？

练习 14：归档最新 run

练习目标：知道如何交付结果。

操作步骤

```
flowctl archive --run latest  
ls reports
```

你应该看到什么

看到 run_xxx.zip。

标准答案/解释

归档文件包含这次运行的日志和报告，适合保存和分享。

进阶思考

为什么归档 run 比只发一个 HTML 报告更完整？

练习 15：用 ask 找日志文档

练习目标：体验内部知识库检索。

操作步骤

```
flowctl ask "怎么看日志"
```

你应该看到什么

返回 docs/log_debugging.md。

标准答案/解释

ask 不是大模型，是关键词检索，但它体现了内部工具支持思路。

进阶思考

如果搜索不到，如何换关键词？

练习 16：用 grep 找 Python 函数

练习目标：学会定位代码入口。

操作步骤

```
grep -n "def cmd_run" scripts/python/flowops/flowctl.py  
grep -n "def run_ip_qa" scripts/python/flowops/flowctl.py
```

你应该看到什么

输出函数所在行号。

标准答案/解释

你不需要从头读完整文件，可以按函数定位主线。

进阶思考

为什么读代码要先找入口函数？

练习 17：识别 report 和 metrics 的区别

练习目标：避免混淆人读报告和机器读指标。

操作步骤

```
cat runs/run_006/reports/signoff.rpt  
cat runs/run_006/metrics/signoff.metrics
```

你应该看到什么

rpt 是自然语言报告，metrics 是 key=value。

标准答案/解释

report 给人看，metrics 给程序比较。

进阶思考

真实项目中为什么需要两种格式？

练习 18：解释 CSV 一行数据

练习目标：理解测试数据每行含义。

操作步骤

```
head -n 2 data/testchip_results.csv
```

你应该看到什么

第一行是表头，第二行是一条测试记录。

标准答案/解释

chip_id 是样本，ip_name 是被测 IP，testcase 是测试项，result 是结果，fail_reason 是失败原因。

进阶思考

如果一个 testcase 失败很多，应该如何优先排查？

练习 19：确认 Git 状态

练习目标：知道项目是否有未提交修改。

操作步骤

```
git status -sb  
git log --oneline -5
```

你应该看到什么

工作区干净，能看到最近 commit。

标准答案/解释

Git 状态干净说明源码和已提交版本一致。

进阶思考

为什么运行结果通常不应该提交到 Git？

练习 20：用 1 分钟讲项目

练习目标：把技术内容变成可沟通表达。

操作步骤

不看文档，口头说出项目主线。

你应该看到什么

应包含 Shell 入口、YAML flow、Tcl steps、Python 编排、runs 输出、manifest 追溯。

标准答案/解释

能讲清楚比背术语更重要。导师关心你是否能理解流程、定位问题、继续学习。

进阶思考

哪些内容你不应该夸大？

错误场景 1: cd 进不去目录

错误现象

```
cd mini-ic-flowops  
No such file or directory
```

这句话是什么意思

系统找不到这个文件夹。

最可能原因

你当前目录不是项目所在目录。

一步步排查

```
pwd  
ls  
ls ~/Downloads  
ls ~/Desktop
```

解决命令

```
cd 正确路径/mini-ic-flowops
```

如何避免再次发生

先用 ls 找到项目，再 cd。

错误场景 2: Permission denied

错误现象

```
./bin/flowctl doctor  
Permission denied
```

这句话是什么意思

文件存在，但 Linux 不允许执行。

最可能原因

没有执行权限。

一步步排查

```
ls -l bin/flowctl
```

解决命令

```
chmod +x bin/flowctl  
./bin/flowctl doctor
```

如何避免再次发生

第一次复制到 Linux 后先 chmod。

错误场景 3: flowctl command not found

错误现象

```
flowctl doctor  
command not found
```

这句话是什么意思

系统找不到 flowctl 命令。

最可能原因

没有 source env.sh 或 PATH 未更新。

一步步排查

```
pwd  
source config/env.sh  
which flowctl
```

解决命令

```
source config/env.sh
```

如何避免再次发生

每次新开终端先 source。

错误场景 4: doctor 提示 tclsh missing

错误现象

```
[WARN] tclsh 未找到
```

这句话是什么意思

系统没有 Tcl 解释器。

最可能原因

Linux 没安装 tcl。

一步步排查

```
which tclsh  
flowctl doctor
```

解决命令

```
sudo apt install -y tcl
```

如何避免再次发生

装好基础工具后再跑。

错误场景 5: python3 missing

错误现象

```
[WARN] python3 未找到
```

这句话是什么意思

系统找不到 Python 3。

最可能原因

Python 未安装或 PATH 不包含 python3。

一步步排查

```
which python3  
python3 --version
```

解决命令

```
sudo apt install -y python3
```

如何避免再次发生

安装 Ubuntu 时确认包含 Python。

错误场景 6: run 后没有 run_001

错误现象

```
ls runs  
run_006
```

这句话是什么意思

不是错误，编号会递增。

最可能原因

你之前已经运行过多次。

一步步排查

```
flowctl status  
ls runs
```

解决命令

使用最新编号即可。

如何避免再次发生

不要固定假设一定是 run_001。

错误场景 7: status 没看到最新运行

错误现象

`flowctl status` 结果为空

这句话是什么意思

还没有任何 run。

最可能原因

没有执行 `flowctl run` 或当前项目目录不对。

一步步排查

```
pwd
ls runs
flowctl run --design alu --flow testchip
```

解决命令

先跑一次 flow。

如何避免再次发生

跑完看终端是否显示运行完成。

错误场景 8: compare 找不到 run

错误现象

找不到 run: run_999

这句话是什么意思

你输入的 run 编号不存在。

最可能原因

编号写错。

一步步排查

```
flowctl status  
ls runs
```

解决命令

```
flowctl compare 正确编号A 正确编号B
```

如何避免再次发生

比较前先查 status。

错误场景 9：qa 失败

错误现象

QA 完成：FAIL

这句话是什么意思

IP 包检查没通过。

最可能原因

缺 README/ip.yaml/RTL/SDC 或版本字段。

一步步排查

```
flowctl qa --design badip  
cat reports/badip_standalone_ip_qa/ip_qa_report.md
```

解决命令

补齐 FAIL 项后重跑 QA。

如何避免再次发生

交付前先跑 QA。

错误场景 10: manifest 看不懂

错误现象

cat manifest.json 输出很多括号

这句话是什么意思

JSON 是结构化文本。

最可能原因

你还不熟悉 JSON 格式。

一步步排查

先找 status/design/flow/git_commit/steps

解决命令

用编辑器格式化查看。

如何避免再次发生

第一遍只看关键字段。

错误场景 11：HTML 报告打不开

错误现象

双击没反应

这句话是什么意思

文件关联或路径问题。

最可能原因

Linux 桌面环境不完整，或你在纯终端里。

一步步排查

```
ls runs/run_006/reports/flow_report.html
```

解决命令

复制到 Windows 用浏览器打开，或看 flow_report.md。

如何避免再次发生

HTML 只是阅读格式，不影响 run 结果。

错误场景 12：source 后路径不对

错误现象

```
FLOWOPS_ROOT=/wrong/path
```

这句话是什么意思

环境变量指向错误目录。

最可能原因

你在错误位置 source。

一步步排查

```
pwd  
cat config/env.sh
```

解决命令

```
cd 项目根目录  
source config/env.sh
```

如何避免再次发生

source 前先确认 pwd。

错误场景 13：YAML 修改后 run 失败

错误现象

YAML 列表项没有所属 key

这句话是什么意思

配置格式不符合解析器要求。

最可能原因

缩进或冒号写错。

一步步排查

```
cat -n config/testchip_flow.yaml
```

解决命令

恢复 steps: 和 - step 格式。

如何避免再次发生

YAML 对缩进敏感，改前备份。

错误场景 14：删除 badip 后 qa 还查 badip

错误现象

找不到设计：badip

这句话是什么意思

badip 目录已不存在。

最可能原因

你已经 `rm -rf designs/badip`。

一步步排查

`ls designs`

解决命令

换成 alu：`flowctl qa --design alu`

如何避免再次发生

清理实验目录后不要再跑 badip。

错误场景 15: GitHub 下载后脚本换行异常

错误现象

```
/usr/bin/env: 'bash\r': No such file
```

这句话是什么意思

Windows CRLF 换行影响 Linux 执行。

最可能原因

文件换行被转换。

一步步排查

```
file bin/flowctl
```

解决命令

```
git config core.autocrlf false 后重新 clone, 或 dos2unix bin/flowctl
```

如何避免再次发生

.gitattributes 已尽量避免此问题。

错误场景 16: archive 没生成 zip

错误现象

还没有任何 run

这句话是什么意思

没有可归档对象。

最可能原因

没有运行过 flow。

一步步排查

`flowctl status`

解决命令

```
flowctl run --design alu --flow testchip  
flowctl archive --run latest
```

如何避免再次发生

归档前先确认有 run。

错误场景 17: ask 搜不到

错误现象

没有找到匹配文档

这句话是什么意思

关键词没匹配到 docs。

最可能原因

问题太长或关键词不在文档。

一步步排查

flowctl ask 日志
flowctl ask QA
flowctl ask manifest

解决命令

换短关键词。

如何避免再次发生

搜索先用核心词。

错误场景 18: cat 文件乱码

错误现象

中文显示异常

这句话是什么意思

终端编码或字体问题。

最可能原因

终端不支持 UTF-8 或字体缺失。

一步步排查

```
locale  
echo $LANG
```

解决命令

设置 UTF-8 终端，或用 VS Code 打开。

如何避免再次发生

Linux 终端建议使用 UTF-8。

错误场景 19: PowerShell 跑 Linux 命令失败

错误现象

`source` 不是内部或外部命令

这句话是什么意思

你在 Windows PowerShell，不是 Linux bash。

最可能原因

命令环境错了。

一步步排查

```
echo $SHELL  
pwd
```

解决命令

进入 Linux 虚拟机终端或 WSL bash。

如何避免再次发生

本项目主练习环境是 Linux。

错误场景 20: tclsh 已安装但还是 fallback

错误现象

flow.log 显示 used Python fallback

这句话是什么意思

Python 没找到 tclsh。

最可能原因

PATH 未包含 tclsh 或安装后终端未刷新。

一步步排查

```
which tclsh  
tclsh <<< 'puts hi'
```

解决命令

重新打开终端，确认 which tclsh。

如何避免再次发生

安装工具后重开终端或刷新 PATH。

错误场景 21：Git status 有很多未跟踪文件

错误现象

```
?? runs/run_xxx
```

这句话是什么意思

生成结果未被 Git 跟踪。

最可能原因

.gitignore 可能未覆盖或你在别的目录。

一步步排查

```
git status -sb  
cat .gitignore
```

解决命令

通常无需提交 runs 结果。

如何避免再次发生

源码和生成结果要分清。

错误场景 22: push 失败

错误现象

Authentication failed

这句话是什么意思

GitHub 认证失败。

最可能原因

gh 未登录或 token 失效。

一步步排查

gh auth status

解决命令

gh auth login

如何避免再次发生

上传前确认 gh auth status。

错误场景 23: run 很快结束怀疑没跑

错误现象

[OK] 运行完成: run_xxx

这句话是什么意思

本项目是模拟流程，所以很快。

最可能原因

没有真实 EDA 工具计算。

一步步排查

```
ls runs/run_xxx/reports  
cat flow.log
```

解决命令

查看报告确认步骤都执行了。

如何避免再次发生

模拟项目重点是流程，不是耗时。

错误场景 24：修改 Tcl 后结果没变

错误现象

compare 无差异

这句话是什么意思

可能走了 Python fallback。

最可能原因

没安装 tclsh 或 fallback 指标未改。

一步步排查

```
flowctl doctor  
grep fallback runs/run_XXX/logs/flow.log
```

解决命令

安装 tcl，或修改 flowctl.py 中 FALLBACK_METRICS。

如何避免再次发生

先确认真实执行路径。

错误场景 25: rm 命令删错担心

错误现象

```
rm -rf designs/badip
```

这句话是什么意思

rm 会删除文件夹。

最可能原因

路径写错可能危险。

一步步排查

```
pwd  
ls designs
```

解决命令

只删除实验目录 badip。

如何避免再次发生

删除前先 ls 确认目标。

错误场景 26: less 退出不了

错误现象

less flow.log 后卡住

这句话是什么意思

less 是分页查看器。

最可能原因

你不知道退出按键。

一步步排查

按 q

解决命令

按 q 退出 less。

如何避免再次发生

记住 less 里 q 退出，/ 搜索。

错误场景 27: grep 没结果

错误现象

```
grep -i fail flow.log 无输出
```

这句话是什么意思

没有匹配文本。

最可能原因

日志里没有 fail 或大小写/路径不对。

一步步排查

```
tail -n 80 flow.log
```

解决命令

换关键词 error、WARN、step。

如何避免再次发生

grep 没输出不一定是错误。

错误场景 28: Word/PDF 找不到

错误现象

ls outputs 无文件

这句话是什么意思

文件可能在宿主 Windows outputs 目录。

最可能原因

你在 Linux 虚拟机内找 Windows 工作区。

一步步排查

```
pwd  
ls
```

解决命令

从 GitHub docs 下载，或复制本地 outputs 文件。

如何避免再次发生

区分 Windows 路径和 Linux 路径。

错误场景 29：运行命令复制了中文引号

错误现象

```
flowctl ask “怎么看日志”
```

这句话是什么意思

中文引号可能导致参数异常。

最可能原因

从富文本复制命令。

一步步排查

手动输入英文双引号

解决命令

```
flowctl ask "怎么看日志"
```

如何避免再次发生

命令里的符号尽量用英文半角。

错误场景 30：不知道问导师什么

错误现象

这个报错了怎么办

这句话是什么意思

信息太少，导师难判断。

最可能原因

没有提供 run、步骤、日志和已检查内容。

一步步排查

```
flowctl status  
tail -n 80 runs/run_xxx/logs/flow.log
```

解决命令

整理：run 编号、失败步骤、第一条错误、已检查内容。

如何避免再次发生

提问要带证据。

术语词典 1-10

术语	一句话解释	本项目哪里出现
Linux	常用于服务器和 EDA 工具运行的操作系统	运行 flowctl、看日志、装 tcl
Terminal	输入命令的窗口	所有命令都在终端执行
Shell	解释命令的程序	bin/flowctl 是 Shell 脚本
Bash	常见 Shell	脚本第一行用 /usr/bin/env bash
PATH	命令搜索路径	source 后 bin 加入 PATH
source	在当前终端加载脚本	source config/env.sh
chmod	修改文件权限	chmod +x bin/flowctl
script	脚本文件	Shell/Tcl/Python 都是脚本
argument	命令参数	--design alu 是参数
option	命令选项	--flow testchip 是选项

术语词典 11-20

术语	一句话解释	本项目哪里出现
flag	开关型选项	--yes 这种通常是 flag
YAML	简洁配置格式	config/testchip_flow.yaml
JSON	结构化数据格式	manifest.json
CSV	表格文本格式	testchip_results.csv
HTML	网页格式	flow_report.html
RTL	寄存器传输级设计代码	designs/alu/rtl/alu.v
Verilog	硬件描述语言	alu.v 是 Verilog
SDC	时序约束文件	alu.sdc
EDA	电子设计自动化	综合、布局布线、签核工具
flow	一串自动化步骤	ref/testchip flow

术语词典 21-30

术语	一句话解释	本项目哪里出现
run	一次流程运行	runs/run_006
log	日志	flow.log
report	报告	synth.rpt、flow_report.html
metrics	结构化指标	signoff.metrics
manifest	运行身份证	manifest.json
hash	文件指纹	sha256
commit	Git 快照	024082d
branch	Git 分支	main
remote	远程仓库	origin
QA	质量检查	flowctl qa

术语词典 31-40

术语	一句话解释	本项目哪里出现
IP	可复用设计模块	alu 示例 IP
testchip	测试芯片流程	testchip_flow.yaml
yield	良率	PASS / total
lot	生产批次	CSV 字段 lot
wafer	晶圆	CSV 字段 wafer
die	晶圆上的芯片单元	die_x/die_y
testcase	测试用例	CSV 字段 testcase
fail_reason	失败原因	timing_margin 等
synth	综合	scripts/tcl/synth.tcl
floorplan	版图规划	floorplan.tcl

术语词典 41-50

术语	一句话解释	本项目哪里出现
place	单元摆放	place.tcl
route	布线	route.tcl
signoff	签核	signoff.tcl
DRC	设计规则检查	route/signoff 报告
LVS	版图网表一致性检查	signoff 报告
WNS	最差时序裕量	synth/signoff 指标
power	功耗	signoff power_mw
area	面积	synth area_um2
cell_count	单元数量	synth 指标
fallback	备用执行路径	没有 tclsh 时 Python 模拟

术语词典 51-60

术语	一句话解释	本项目哪里出现
subprocess	Python 调外部命令	调用 tclsh/git
argparse	Python 命令行解析库	build_parser
function	函数	cmd_run/run_ip_qa
dict	字典	metrics 数据
list	列表	steps 列表
Path	Python 路径对象	ROOT / config
stdout	标准输出	命令正常输出
stderr	标准错误	错误输出
exit code	退出码	0 成功，非 0 失败
precheck	输入预检查	检查 README/RTL/SDC

术语词典 61-70

术语	一句话解释	本项目哪里出现
README	说明文档	IP 包说明
ip.yaml	IP 元数据	name/version/top_module
top_module	顶层模块	alu
constraints	约束文件夹	alu.sdc
hard-coded path	写死路径	QA 会警告
empty file	空文件	QA 会检查
archive	归档	flowctl archive
zip	压缩包	reports/run_xxx.zip
GitHub	远程代码平台	osifj/mini-ic-flowops
origin	默认远程名	git remote -v

术语词典 71-80

术语	一句话解释	本项目哪里出现
main	默认分支名	GitHub 主分支
status	状态	PASS/FAIL/RUNNING
PASS	通过	run 成功
FAIL	失败	QA 或 flow 失败
WARN	警告	不一定阻塞但要注意
HTML report	网页报告	可浏览器打开
Markdown	轻量文档格式	.md 文件
key=value	键值格式	metrics 文件格式
config	配置	flow yaml/env.sh
environment variable	环境变量	FLOWOPS_ROOT/PATH

术语词典 81-90

术语	一句话解释	本项目哪里出现
license	商业工具授权	真实 EDA 环境常见
PDK	工艺设计套件	真实公司内部资产
Reference Flow	参考流程	ref_flow.yaml
Testchip Flow	测试芯片流程	testchip_flow.yaml
tool version	工具版本	manifest 记录
input hash	输入文件指纹	manifest inputs.files
reproduce	复现	用相同输入和版本重跑
debug	排查问题	看日志/报告/manifest
grep	搜索文本	grep -i fail flow.log
tail	看文件末尾	tail -n 80

术语词典 91-100

术语	一句话解释	本项目哪里出现
cat	打印文件	cat config/testchip_flow.yaml
less	分页看文件	less flow.log
mkdir	创建目录	run 目录创建
rm	删除文件	制造 QA 失败时使用
cp	复制文件	复制 badip
working tree	Git 工作区	git status
clean	无未提交修改	git status 干净
push	推送到远程	git push
clone	克隆仓库	可从 GitHub 拉项目
AI retrieval	AI 检索	flowctl ask 的简化版

术语词典 101-101

术语	一句话解释	本项目哪里出现
RAG	检索增强生成	未来 AI 工具扩展方向

附录：面试/实习 50 题速查版

你说 50
题不必扩写，所以本版只保留速查。详细版本已经在上一版教材中存在。这里用于临考前快速扫一遍。

分类	题目范围	复习重点
Linux 基础	为什么用 Linux、路径、权限、source、PATH、tail	能在终端独立跑项目和看日志
Shell/Tcl/Python	三者分工、入口、Tcl argv、Python argparse/subprocess	能解释脚本链路
Flow 自动化	flow、ref/testchip、precheck、metrics、compare	能解释自动化流程
IP QA	README、ip.yaml、RTL、SDC、硬编码路径、PASS/WARN/FAIL	能做交付包检查
测试数据分析	CSV、yield、fail_by_ip、fail_reason、JSON/Markdown	能读分析报告
Git/版本管理	status、commit、hash、manifest、GitHub	能追溯结果变化
AI 工具部署	数据安全、flowctl ask、检索和 RAG	不要夸大成完整大模型

最终复习：你应该全面了解项目到什么程度

看完这本书后，你不需要说自己精通 IC 设计，但应该能清楚讲出这个项目的结构和工作流。

- 用户输入 `flowctl run` 后，Shell 入口会调用 Python 主程序。
- Python 读取 YAML flow 配置，创建 `runs/run_xxx`。
- `precheck` 检查输入，Tcl steps 生成模拟 EDA 报告。
- `ip_qa` 检查 IP 交付包，`data_collect` 分析测试 CSV。
- `report` 生成总报告，`manifest` 记录版本、工具、输入和步骤。
- 日志用于排错，`metrics` 用于比较，`reports` 用于阅读，`manifest` 用于追溯。

如果你能把上面这段用自己的话讲出来，并独立完成

`run`、`status`、`qa`、`compare`、`archive`，实习前的脚本和流程基础就够用了。